Last Name: First Name:

Matriculation Number:

Retake Exam
Artificial Intelligence 1

October 9, 2023

	To be used for grading, do not write here													
prob.	1.1	2.1	2.2	3.1	4.1	4.2	5.1	5.2	5.3	6.1	7.1	7.2	Sum	grade
total	10	10	9	6	7	7	6	8	7	8	10	6	94	
reached														

In the course Artificial Intelligence I/II we award bonus points for the first student who reports a factual error in an old exam. (Please report spelling/formatting errors as well.)

1 Prolog

Problem 1.1 (Prolog in Prolog)
Consider the following *Prolog program* that represents *Prolog* in *Prolog*, i.e. *Prolog terms*, *literals*, and *clauses* are represented as *Prolog terms*:

```
isTerm(pterm(F,ARGS)) :- string(F), isTermList(ARGS).
isTerm(pvar(X)) :- string(X).

isTermList([]).
isTermList([H|T]) :- isTerm(H), isTermList(T).

isLiteral(plit(P,ARGS)) :- string(P), isTermList(ARGS).

isLiteralList([]).
isLiteralList([H|T]) :- isLiteral(H), isLiteralList(T).

isClause(pclause(H,B)) :- isLiteral(H), isLiteralList(B).
```

Here `string` is a built-in predicate that succeeds if its argument is a string.

1. Explain intuitively what the *predicate* `isClause(pclause(H,B))` *computes*? 2 pt

   Solution: It checks whether its *argument* represents a *Prolog clause* with d *head* `H` and *body* `B`.

2. Write the *Prolog clause* `isNat(succ(N)) :- isNat(N)` as a *Prolog term* relative to the above *program* (i.e., such that `isClause` succeeds for it). 3 pt

   Solution:

   ```
   pclause(plit("isNat", [pterm("succ",[pvar("N")])]),
           [plit("isNat", [pvar("N")])]).
   ```

3. Assume that the *Prolog term C* contains no *free variables*. How is the result of the *query* `isClause(C)` affected by exchanging the lines 4 and 5? 2 pt

   Solution: It is not affected.

4. Extend the *program* above with a *unary Prolog predicate* `isProgram` that succeeds if its *argument* is of the form `pprog(P)` where *P* is a *list* of *clauses*. 3 pt

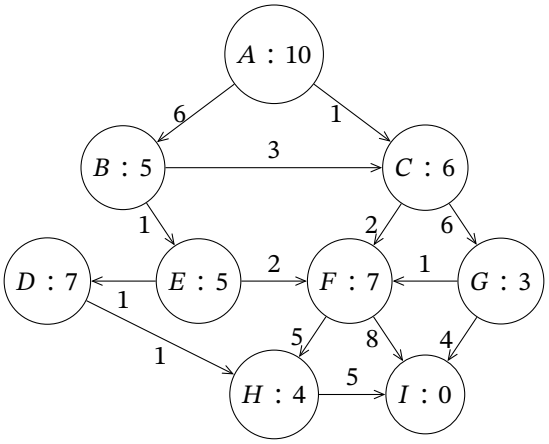
   Solution:

   ```
   isClauseList([]).
   isClauseList([H|T]) :- isClause(H), isClauseList(T).

   isProgram(pprog(C)) :- isClauseList(C).
   ```

2 Search

Problem 2.1 (Search Algorithms)
Consider the following *directed graph*:



Every *node* is *labeled* with $n : h(n)$ where n is the identifier of the *node* and $h(n)$ is the *heuristic* for estimating the *cost* from n to a *goal node*. Every *edge* is *labeled* with its actual *cost*.

1. Assume that I is the *goal node*. Argue whether or not the *heuristic* is *admissible*. 2 pt

   Solution: It is not *admissible*: The *cost* from D to the *goal* is $1+5=6<7=h(D)$, and a *heuristic* must not overestimate that *cost*.

Now assume you have already *expanded* the *node* A. List the **next** 4 ***nodes* (i.e., excluding** A**)** that will be *expanded* using the respective *algorithm*. If there is a tie, break it using alphabetical order.

2. *Depth-first search* 1 pt

   Solution: B, C, F, H

3. *Breadth-first search* 1 pt

   Solution: B, C, C, E (B, C, E, F can also be seen as correct)

4. *uniform-cost search* 2 pt

   Solution: C, F, B, E

5. *greedy search* 2 pt

   Solution: B, E, C, G

6. *A*-search* 2 pt

Solution: C, F, G, B

Problem 2.2 (Search Problems)
Consider the *search problem* $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{I}, \mathcal{G} \rangle$ where

- ☐ $\mathcal{S} = \mathbb{Z} \times \mathbb{Z}$
- ☐ $\mathcal{A} = \{R, S, M\}$
- ☐ $\mathcal{T}$ is given by
  - ☐ $\mathcal{T}(R, (x, y), =)\{(x, 0), (0, y)\}$
  - ☐ $\mathcal{T}(S, (x, y), =)\{(y, x)\}$
  - ☐ $\mathcal{T}(M, (x, y), =)\{(x + 1, y)\}$
- ☐ $\mathcal{I} = \{(0, 0)\}$
- ☐ $\mathcal{G} = \{(3, 3)\}$

1. Tick the box of the part of the definition that makes this *problem fully observable*. 1 pt

Solution: the one for I

2. Give the possible *states* resulting from applying the *action* sequence M, R, M to the *initial state*. 2 pt

Solution: $(2, 0), (1, 0)$

3. Which *states* are *reachable* from the *initial state*? 2 pt

Solution: all *states* with non-negative coordinates

4. Give a *solution* of minimal *length* to this *problem*. 2 pt

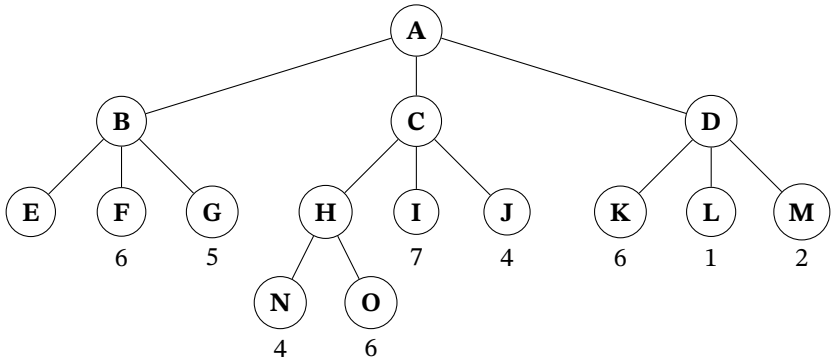
Solution: M, M, M, S, M, M, M

5. Assume we use $h((x, y)) = 1/(1+x+y)$ as a *heuristic*. Which *action* will a *greedy search algorithm* choose for 2 pt
   1. the first step
   2. the subsequent second step

Solution: M, M

3 Adversarial Search

Problem 3.1 (Minimax)
Consider the following *minimax* game tree for the **maximizing** player's turn. The *values* at the *leaves* are the static *evaluation function values* of those *state*; some of those *values* are currently missing.



1. Label the *nodes* H and C with their *minimaxs utility functions*. 2 pt

   Solution: H: 6, C: 4

2. If possible, label the *node* **E** with an *evaluation function value* that results in the player definitely choosing move **C** (no matter how ties are broken). 2 pt
   Otherwise, argue why that is impossible.

   Solution: Any label < 4.

3. Now assume **E** is *labeled* with 5, and we use $\alpha\beta$*-pruning*. We expand *child nodes* in alphabetical order. Which nodes would be *pruned*? 2 pt

   Solution: M

4 Constraint Satisfaction/Propagation

Problem 4.1 (Modeling) 7 pt
You want to schedule a tournament in which teams A, B, C, D play each other once. The six games must take place over the next 3 days. But team A must not play twice on the same day. Team B is only available for the next 2 days. And team C wants to play against D a day before playing against anybody else.

Model this problem as a *constraint network* $\langle V, D, C\rangle$. Explain how the *solutions* correspond to the possible match schedules.

Solution: E.g.,
$V = \{AB, AC, AD, BC, BD, CD\}$

$D_{XY} = \{1,2,3\}$ for all $XY \in V$
Constraints in C:

- A-matches on different days: $AB \neq AC$, $AB \neq AD$, $AC \neq AD$
- B-matches on first two days: $AB \leq 2$, $BC \leq 2$, $BD \leq 2$
- CD-match before other C-matches: $CD < AC$, $CD < BC$

Explanation: For a *solution* s, the match XY is played on day $s(XY)$.

Problem 4.2 (Modeling) 7 pt
You want to schedule a tournament in which teams A, B, C, D play each other once. The six games must take place over the next 3 days. But team A must not play twice on the same day. Team B is only available for the next 2 days. And team C wants to play against D a day before playing against anybody else.

Model this problem as a *constraint network* $\langle V, D, C\rangle$. Explain how the *solutions* correspond to the possible match schedules.

Solution: E.g.,
$V = \{AB, AC, AD, BC, BD, CD\}$
$D_{XY} = \{1,2,3\}$ for all $XY \in V$
Constraints in C:

- A-matches on different days: $AB \neq AC$, $AB \neq AD$, $AC \neq AD$
- B-matches on first two days: $AB \leq 2$, $BC \leq 2$, $BD \leq 2$
- CD-match before other C-matches: $CD < AC$, $CD < BC$

Explanation: For a *solution* s, the match XY is played on day $s(XY)$.

---

5 Logic

Problem 5.1 (Propositional Logic)
We use the *propositional variables* X, Y, and Z. Consider the *formula* A given by

$$X \wedge (Y \Rightarrow Z) \Rightarrow \neg(X \wedge Y)$$

1. Give a *satisfying assignment* σ and a *falsifying assignment* φ for A. 3 pt

---

Solution: The *satisfying* and *falsifying assignments* can be read off the following table. $\varphi(X) = \varphi(Y) = \varphi(Z) = 1$ and σ any other *assignment*.

X	Y	Z	*interpretation* of A
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

2. Which (if any) of the *formulae* A and $\neg A$ is *valid*? 1 pt

Solution: Neither (because A and $\neg A$ are both are *satisfiable* and *falsifiable*).

3. Give the shortest *formula* in *CNF* that is *equivalent* to $A \Rightarrow A$. 2 pt

Solution: T (because the *formula* is *valid*)

Problem 5.2 (Predicate Logic)
Consider the following *first-order signature* Σ:

- *binary function constant* f
- *unary predicate constant* p

1. Give a *closed formula* over Σ that uses all its constants. 2 pt

Solution: E.g., $\forall x.p(f(x,x))$

2. Give a *first-order model* for Σ 2 pt

Solution: E.g., *universe* $\mathbb{N}$, $\mathcal{I}(f)(u,v) = u+v$, $\mathcal{I}(p) = \{0\}$.

3. Consider the *formula* $A = p(x) \wedge p(y)$ and assume a *first-order model* with *universe* $\mathbb{N}$ and $\mathcal{I}(p) = \{0\}$.
Give an *assignment* α such that $\mathcal{I}_\alpha(A) = \mathsf{T}$. 2 pt

Solution: $\alpha(x) = \alpha(y) = 0$

4. Prove or refute the following statement: A *first-order model* that *satisfies* $\forall x.p(x)$, *satisfies* all *formulae*. 2 pt

Solution: False. Any *model* that satisfies $\forall x.p(x)$ (of which there are many) does not satisfy F.

Problem 5.3 (Proving in Tableau Calculus) 7 pt
We use the *propositional variables* P, Q, and R and the following abbreviations

$$A := Q \wedge (P \Rightarrow Q)$$

$$B := P \Rightarrow A$$

$$C := P \Rightarrow R$$

Using the *tableau calculus*, find all *falsifying f* or the *formula* $C \Rightarrow B$.

Solution:

$$\frac{(C \Rightarrow B)^{\mathsf{F}}}{\begin{array}{c} C^{\mathsf{T}} \\ B^{\mathsf{F}} \\ P^{\mathsf{T}} \\ A^{\mathsf{F}} \end{array}}$$

P^{F}	R^{T}	
	Q^{F}	$(P \Rightarrow Q)^{\mathsf{F}}$
close on P	open: $P^{\mathsf{T}}, Q^{\mathsf{F}}, R^{\mathsf{T}}$	P^{T}
		Q^{F}
		open: $P^{\mathsf{T}}, Q^{\mathsf{F}}, R^{\mathsf{T}}$

So the only falsifying assignment φ is given by $\varphi(P) = \varphi(R) = \mathsf{T}$ and $\varphi(Q) = \mathsf{F}$

---

6 Knowledge Representation

Problem 6.1 (Description Logic)
Consider the following $\mathcal{ALC}$ setting:

- concepts: pizza, icecream, food, topping
- relations: canHaveTopping
- individuals: margarita, vanilla, ham, syrup

You may abbreviate every concept/relation/individual by its first letter.

1. Give the $\mathcal{ALC}$ *ABox* with *concept axioms* that model common sense knowledge (e.g., we do not put vanilla or syrup on pizza even though it is technically possible). 2 pt

   ---

   Solution: $m : p, v : i, h : t, s : t, p\, c\, h, i\, c\, s$

   ---

2. Give an $\mathcal{ALC}$ *TBox* with two *concept axioms* expressing the following: 2 pt
   - All food is pizza or icecream.
   - Only pizza can have toppings.

   Solution: $f \sqsubseteq p \sqcup i, \exists c.t \sqsubseteq p$

   ---

3. Give an $\mathcal{ALC}$ *TBox* in which the *concept* pizza is *inconsistent*. 2 pt

   ---

   Solution: E.g., $p \sqsubseteq \bot$

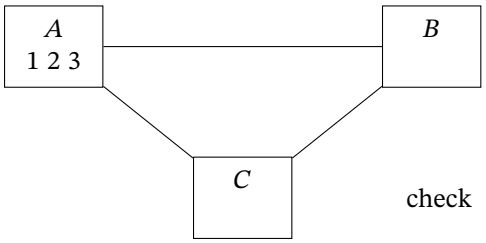
   ---

4. Give the result of translating the following *ALC formula* to *first-order logic*: $(\forall c.i) \sqsubseteq (p \sqcap t)$ 2 pt

Solution: $\forall x.(\forall y.c(x \Rightarrow y) \Rightarrow i(y)) \Rightarrow p(x) \wedge t(x)$

7 Planning

Problem 7.1 (STRIPS)
Consider a machine that processes objects $Obj = \{1, 2, 3\}$, which can be at location A, B, or C. Currently all objects are at location A and **unchecked** and **assembled**. Eventually all objects are needed in location A and **checked** and **assembled**.
At location B, objects can be assembled or disassembled. At location C, disassembled objects can be checked.
A transport system is available that can move **exactly two objects at a time** from one location to any other location.



We formalize this problem as a *STRIPS task* where the *facts* are

- position $l \in \{A, B, C\}f$ of object $o \in Obj$: at(l, o)
- state of object $o \in Obj$: isCh(o) (checked), isAss(o) (assembled), and isDis(o) (disassembled)

and the *actions* are given by table below for any $l, m \in \{A, B, C\}$ and $o, p \in Obj$

action	precondition	add list	delete list
move(l, m, o, p)	at(l, o), at(l, p)	at(m, o), at(m, p)	at(l, o), at(l, p)
assemble(o)	at(B, o)	isAss(o)	isDis(o)
disassemble(o)	at(B, o)	isDis(o)	isAss(o)
check(o)	at(C, o), isDis(o)	isCh(o)	

1. Give the *initial state* I and the *goal state* G. 2 pt

Solution: *initial state*: at(A, o) and isAss(o) for all $o \in Obj$,
goal state: at(A, o), isCh(o) and isAss(o) for all $o \in Obj$

2. Give the *state* after applying the two *actions* move$(A, B, 1, 2)$ followed by disassemble(1). 2 pt

Solution: at$(A, 3)$, at$(B, 1)$, at$(B, 2)$, isDis(1), isAss(2), isAss(3)

3. Give the value $h^*(I)$. 2 pt

Solution: 17 (3 *actions* per object for disassemble, check, assemble, as well as 2 sequences of 4 move actions $A - B - C - B - A$ for pairs of objects)

4. Give the value $h^+(I)$. 2 pt

Solution: 10. (With the *delete heuristic*, objects do not have to re-assembled or moved back. So only 2 actions are needed per object instead of 3 and only 2 moves per pair of objects.)

5. Consider the *heuristic* h that *computes* $h(s)$ as $2a + d + 3$ where a is the number of unchecked assembled and d the number of unchecked disassembled objects in *state* s. Argue whether h is *admissible*. 2 pt

Solution: It is not *admissible*. A counter-example is the *goal state* g where $h^*(g) = 0 < 3 = h(g)$ in violation of the *admissibility condition*.

Problem 7.2 (Complexity) 6 pt

1. What is the difference between satisficing and optimal planning?

Solution: Satisficing planning searches for any plan. Optimal planning for one with minimal length.

2. Give the complexity classes of deciding the existence of
   3. a plan for a STRIPS problem

   Solution: PSPACE (which is the same as NPSPACE).

   4. a plan of a certain maximal length for a STRIPS problem

   Solution: PSPACE (which is the same as NPSPACE).

   Here "complexity classes" are the usual named classes such as P, NP, etc.
5. Now we consider only STRIPS problems in which all delete lists are empty.
   Prove that deciding if a plan exists is in NP.

Solution: Without delete list, an action cannot decrease the number of facts in the state. Actions that do not change the state are redundant. Therefore, the number of facts in the goal is an upper bound for the length of an optimal plan. Finding such a plan non-deterministically is bounded by the size of the problem, and therefore definitely polynomial in that size. That proves the problem is in NP.